

Assignment 4 of MATP4820

Izaak King

February 23, 2026

Problem 1

Consider the unconstrained quadratic minimization problem:

$$\underset{\mathbf{x}}{\text{minimize}} f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x} \quad (\text{QuadMin})$$

where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a symmetric positive definite matrix, and $\mathbf{b} \in \mathbb{R}^n$.

1. Let $\mathbf{A} = \begin{bmatrix} 4 & -2 \\ -2 & 3 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. Set the initial vector $\mathbf{x}^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. Do by hand the conjugate gradient method to find the **exact solution**.

First, take $x^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Iteration 0:

$$r^{(1)} = Ax^{(1)} - b = -b = \begin{bmatrix} -2 \\ -1 \end{bmatrix}$$

$$p^{(1)} = -r^{(1)} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Iteration 1: $k = 1$

$$Ap^{(1)} = \begin{bmatrix} 4 & -2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ -1 \end{bmatrix}$$

$$\alpha^{(1)} = -\frac{\langle p^{(k)}, r^{(k)} \rangle}{\langle p^{(1)}, Ap^{(1)} \rangle} = -\frac{\left\langle \begin{bmatrix} -2 \\ -1 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix} \right\rangle}{\left\langle \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 6 \\ -1 \end{bmatrix} \right\rangle} = -\left(-\frac{5}{11}\right) = \frac{5}{11}$$

$$x^{(2)} = x^{(1)} + \alpha^{(1)}p^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \frac{5}{11} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{10}{11} \\ \frac{5}{11} \end{bmatrix}$$

$$r^{(2)} = Ax^{(2)} - b = \begin{bmatrix} 4 & -2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} \frac{10}{11} \\ \frac{5}{11} \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{30}{11} \\ -\frac{5}{11} \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{8}{11} \\ -\frac{16}{11} \end{bmatrix}$$

$$\beta^{(2)} = \frac{\langle r^{(2)}, Ap^{(1)} \rangle}{\langle p^{(1)}, Ap^{(1)} \rangle} = \frac{\left\langle \begin{bmatrix} \frac{8}{11} \\ -\frac{16}{11} \end{bmatrix}, \begin{bmatrix} 6 \\ -1 \end{bmatrix} \right\rangle}{\left\langle \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 6 \\ -1 \end{bmatrix} \right\rangle} = \frac{\frac{64}{11}}{11} = \frac{64}{121}$$

$$p^{(2)} = -r^{(2)} + \beta^{(2)}p^{(1)} = -\begin{bmatrix} \frac{8}{11} \\ -\frac{16}{11} \end{bmatrix} + \frac{64}{121} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{40}{121} \\ \frac{240}{121} \end{bmatrix}$$

Iteration 2: k = 2

$$Ap^{(2)} = \begin{bmatrix} 4 & -2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} \frac{40}{121} \\ \frac{240}{121} \end{bmatrix} = \begin{bmatrix} -\frac{320}{121} \\ \frac{640}{121} \end{bmatrix}$$

$$\alpha^{(2)} = -\frac{\langle p^{(2)}, r^{(2)} \rangle}{\langle p^{(2)}, Ap^{(2)} \rangle} = \frac{\left\langle \begin{bmatrix} \frac{40}{121} \\ \frac{240}{121} \end{bmatrix}, \begin{bmatrix} \frac{8}{11} \\ -\frac{16}{11} \end{bmatrix} \right\rangle}{\left\langle \begin{bmatrix} \frac{40}{121} \\ \frac{240}{121} \end{bmatrix}, \begin{bmatrix} -\frac{320}{121} \\ \frac{640}{121} \end{bmatrix} \right\rangle} = -\frac{-\frac{3520}{1331}}{\frac{140800}{14641}} = \frac{11}{40}$$

$$x^{(3)} = x^{(2)} + \alpha^{(2)}p^{(2)} = \begin{bmatrix} \frac{10}{11} \\ \frac{5}{11} \end{bmatrix} + \frac{11}{40} \begin{bmatrix} \frac{40}{121} \\ \frac{240}{121} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$r^{(3)} = Ax^{(3)} - b = \begin{bmatrix} 4 & -2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Thus, we have $r^{(3)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, so we have found the exact solution and we stop iterating.

We get the exact solution of $x^{(3)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

-
2. Use the instructor's provided file `quadMin_cg.m` to write a Matlab function `quadMin_cg` with input **A**, **b**, initial vector **x0**, and tolerance **tol**, and with stopping condition $\|\nabla f(\mathbf{x}^k)\| \leq \text{tol}$. Also test your function by running the provided test file `test_cg.m` and compare to the instructor's function. Print your code and the results you get. Make discussions on what you observe for the difference between your steepest gradient method solver (from the 3rd assignment) and your conjugate gradient method solver.

Conjugate Gradient Method (`quadMin_cg.m`) Solver

```
function [x, hist_res] = quadMin_cg(A,b,x0,tol)

% conjugate gradient method for solving
% min_x 0.5*x'*A*x - b'*x

% get the size of the problem
n = length(b);

x = x0;

% compute vector r, i.e., gradient of the objective
r = A * x - b;

% set the first p vector as negative gradient
p = -r;

% evaluate the norm of gradient
res = norm(r);

% save the value of res
hist_res = res;

while res > tol

    % compute A * p once

    Apk = A * p;
```

```
% compute <p, Apk> once

pDotApk = dot(p, Apk);

% compute alpha

alpha = - dot(p, r) / pDotApk;

% update x

x = x + alpha * p;

% update r

r = A * x - b;

% compute beta

beta = dot(r, Apk) / pDotApk;

% obtain the new p vector

p = -r + beta * p;

% evaluate the norm of residual vector r
res = norm(r);

% save the value of res
hist_res = [hist_res; res];
end

end
```

test_cg.m Output

```
>> test_cg
Results by student code
Total running time is 0.0020
Final objective value is -4551.8686
Results by Instructor code
Total running time is 0.0014
Final objective value is -4551.8686
Results by Student GD code
Total running time is 0.0269
Final objective value is -4551.8686
Results by Instructor GD code
Total running time is 0.0281
Final objective value is -4551.8686
```

NOTE: For comparisons, we will use the performance of both algorithms I created, rather than the instructor's

CG Method Plot

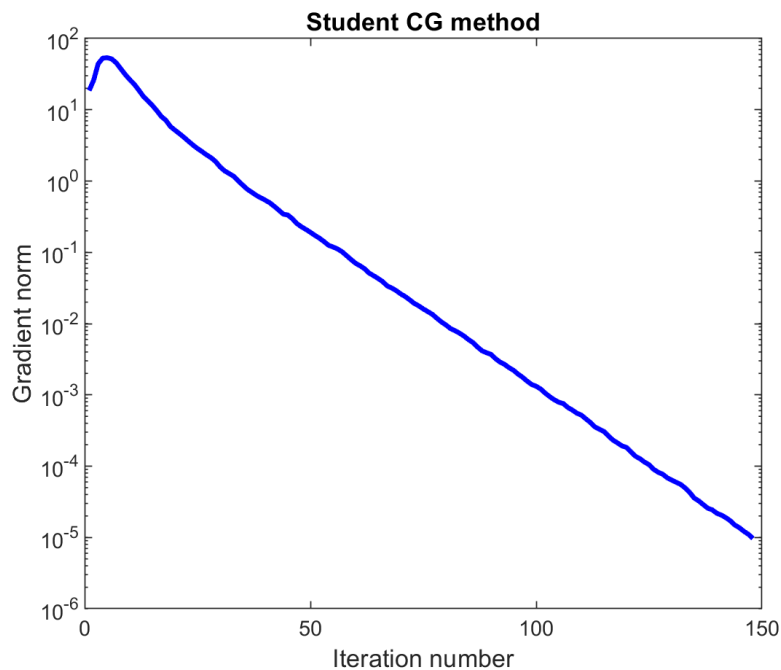


Figure 1: Plot outputted by running Conjugate Gradient method.

GD Method Plot

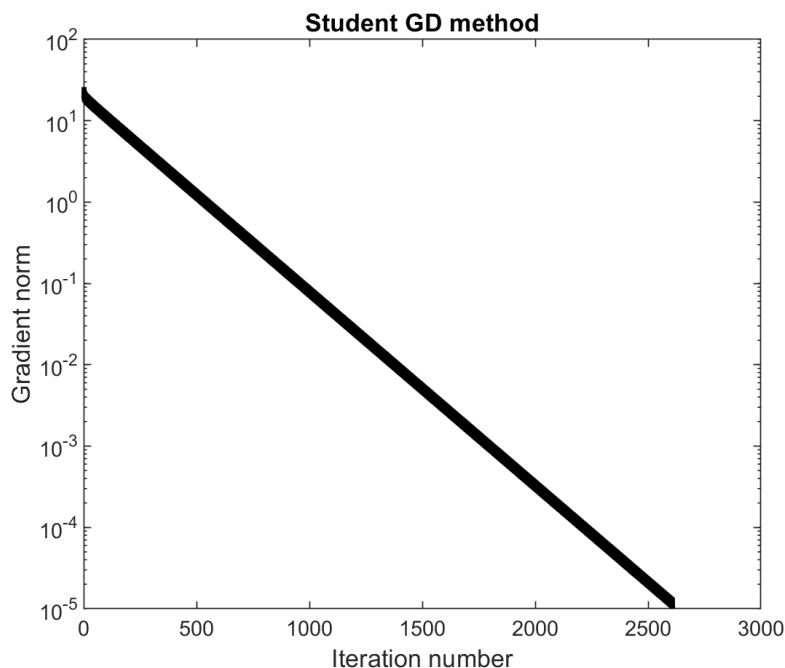


Figure 2: Plot outputted by running Gradient Descent method.

Based on the outputs from our testing, as well as the plots of total iterations, it is clear the Conjugate Gradient method performs much better than the Gradient Descent Method with this test. The running time is much faster, with Conjugate Gradient taking 0.0020 and Gradient Descent taking 0.0281. Similarly, Conjugate Gradient takes about 150 iterations while Gradient Descent takes over 2500, showcasing Conjugate Gradient's significant improvement.

Problem 2

For ill-conditioned problems, the conjugate gradient (CG) method may converge very slowly, and a better method is the preconditioned conjugate gradient (PCG) method. Consider the quadratic minimization problem in (QuadMin) stated in Problem 1. Write a solver for (QuadMin) by the PCG method. We will use a non-singular triangular matrix $\mathbf{C}$ as the preconditioner.

1. Let $\mathbf{C} \in \mathbb{R}^{n \times n}$ be a non-singular upper triangular matrix. Write a solver `pcg_linsolv` for the linear system $\mathbf{C}^\top \mathbf{C} \mathbf{y} = \mathbf{r}$. Treat $\mathbf{C}$ and $\mathbf{r}$ as the input of the solver. Recall that to solve the linear system, we can first solve $\mathbf{C}^\top \mathbf{z} = \mathbf{r}$ by forward substitution and then solve $\mathbf{C} \mathbf{y} = \mathbf{z}$ by back substitution. [Note: you will need to use for-loop in your solver. You CANNOT use MATLAB's solver to directly solve the linear system.]

`pcg_linsolv` Function

```
function [y] = pcg_linsolv(C, r)

    % Initialize C^T, solution vectors z and y, and matrix size n
    CT = transpose(C);
    n = size(CT, 1);
    z = zeros(n, 1);
    y = zeros(n, 1);

    % Forward substitution
    for i = 1:n

        % Add known values up until diagonal
        summation = 0;
        for j = 1:i-1
            summation = summation + CT(i, j) * z(j);
        end

        % Get current diagonal value
        z(i) = (r(i) - summation) / CT(i, i);
    end
```

```
% Backward substitution
for i = n:-1:1

    % Add known values up until diagonal
    summation = 0;
    for j = i + 1:n
        summation = summation + C(i, j) * y(j);
    end

    % Get current diagonal value
    y(i) = (z(i) - summation) / C(i, i);
end
end
```

-
2. Write a Matlab function `quadMin_pcg` with input **A**, **C**, and **b**, where **C** is a non-singular upper triangular matrix, used as the conditioning matrix. Within your PCG solver, use `pcg_linsolv` that you develop in the previous question as the linear solver for solving linear systems in the form of $\mathbf{C}^T \mathbf{C} \mathbf{y} = \mathbf{r}$. Also test your function by running the provided test file `test_pcg.m` and compare to the instructor's function. Print your code and the results you get. [**Hint:** to debug your PCG solver, you can use MATLAB's linear system solver to directly solve $\mathbf{C}^T \mathbf{C} \mathbf{y} = \mathbf{r}$. But your final version of your PCG solver MUST use `pcg_linsolv` that you develop.]

Preconditioned Conjugate Gradient (`quadMin_pcg`) Function

```
function [x, hist_res] = quadMin_pcg(A,C,b,x0,tol)

% conjugate gradient method for solving
% min_x 0.5*x'*A*x - b'*x

% get the size of the problem
n = length(b);

x = x0;

% compute vector r, i.e., gradient of the objective
r = A * x - b;

% compute first y
y = pcg_linsolv(C, r);

% set the first p vector
p = -y;

% evaluate the norm of gradient
res = norm(r);

% save the value of res
hist_res = res;
```

```
while res > tol

    % compute rk dot yk once

    rDoty = dot(r, y);

    % compute A * p once

    Ap = A*p;

    % compute alpha

    alpha = rDoty / dot(p, Ap);

    % update x

    x = x + alpha*p;

    % update r

    r = r + alpha * Ap;

    % compute updated y once

    yNext = pcg_linsolv(C, r);

    % compute beta

    beta = dot(r, yNext) / rDoty;

    % obtain the new p vector

    p = -yNext + beta * p;

    % update y

    y = yNext;
```

```
% evaluate the norm of residual vector r
res = norm(r);

% save the value of res
hist_res = [hist_res; res];
end

end
```

test_pcg.m Output

```
>> test_pcg
Student solver: Total iteration is 130
Total running time is 2.8527
Final objective value is -1.4159
Instructor pcg solver: Total iteration is 130
Total running time is 0.7172
Final objective value is -1.4159
Instructor cg solver: Total iteration is 287
Final objective value is -1.4159
```

Thus we see the Preconditioned Conjugate Gradient Method arrives at the final objective value in less than half the iterations as the instructor's Conjugate Gradient solver. Our PCG method takes 130 total iterations, while the CG method takes 287 iterations to arrive at the same final objective value.

Problem 3

Let $f(x) = x^2 + e^{-x} + 2 \cos x, x \in \mathbb{R}$.

1. Prove that this function is convex and show by graph that this function has a minimizer.

Proof. First, to prove $f(x)$ is convex, we will show $f''(x) \geq 0$ for any $x \in \mathbb{R}$

Now with

$$f(x) = x^2 + e^{-x} + 2 \cos x$$

$$f'(x) = 2x - e^{-x} - 2 \sin x$$

$$f''(x) = 2 + e^{-x} - 2 \cos x$$

Thus, we must show

$$f''(x) = 2 + e^{-x} - 2 \cos x \geq 0 \text{ for all } x \in \mathbb{R}$$

First note,

$$-1 \leq \cos x \leq 1 \implies -2 \leq 2 \cos x \leq 2$$

Then,

$$2 + e^{-x} - 2 \leq 2 + e^{-x} - 2 \cos x \leq 2 + e^{-x} + 2 \implies e^{-x} \leq 2 + e^{-x} - 2 \cos x$$

Now, $e^{-x} > 0$ for all $x \in \mathbb{R}$. Thus,

$$f''(x) = 2 + e^{-x} - 2 \cos x \geq e^{-x} > 0 \text{ for all } x \in \mathbb{R}$$

Therefore, $f''(x) \geq 0$ for all $x \in \mathbb{R}$, and $f(x)$ is convex.

□

Now, showing by graph $f(x)$ has a minimum

Hand-Drawn Graph

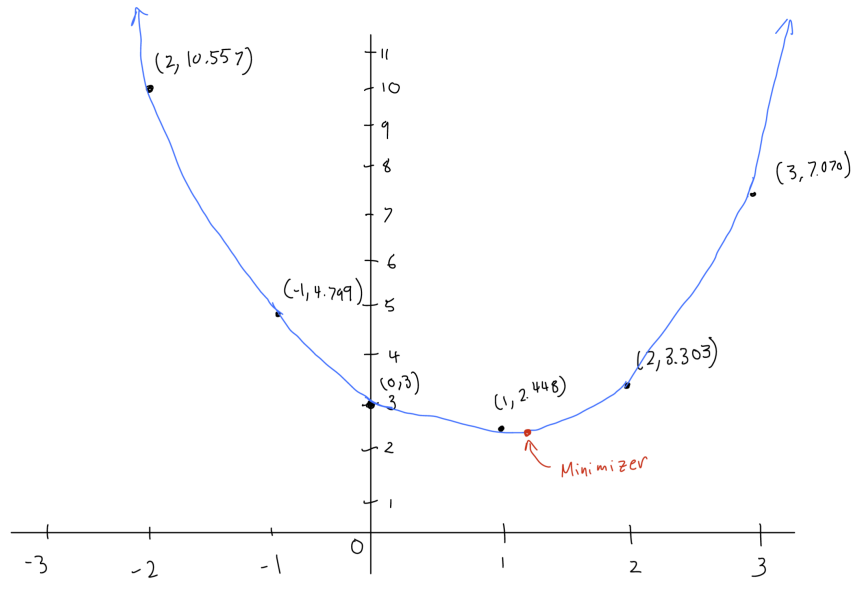


Figure 3: Hand-drawn graph of $f(x)$.

Computed Graph

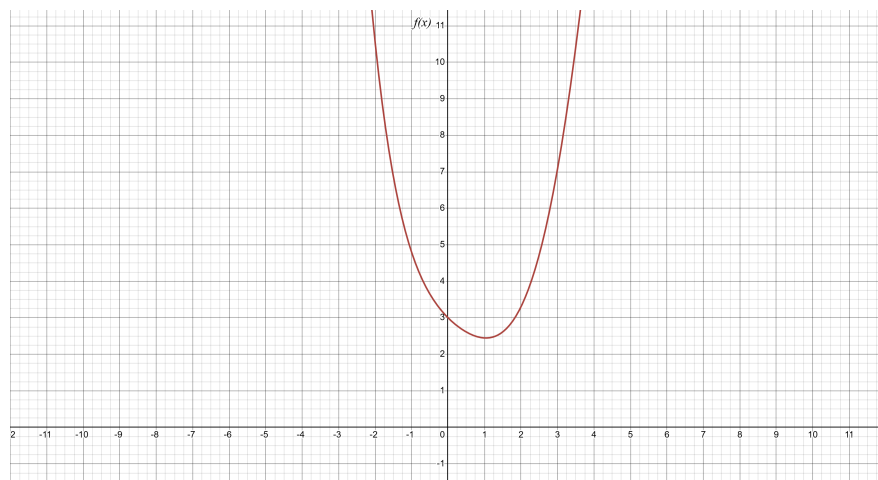


Figure 4: Computed graph of $f(x)$.

Thus in both graphs, hand-drawn and computed, we see $f(x)$ has a clear minimizer value occurring just past $x = 1$. Since the graph continues to strictly increase in both directions to the left and right of this minimizer, we see this a global minimum of $f(x)$.

-
2. Start from $x^{(0)} = 2$ and do 3 iterations of the Newton's method to find an approximate minimizer of $f(x)$ by hand (calculator is allowed).

First, we have

$$f(x) = x^2 + e^{-x} + 2 \cos x$$

$$f'(x) = 2x - e^{-x} - 2 \sin x$$

$$f''(x) = 2 + e^{-x} - 2 \cos x$$

We will use Newton's iteration:

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

Iteration 1:

$$x^{(1)} = x^{(0)} - \frac{f'(x^{(0)})}{f''(x^{(0)})} = 2 - \frac{(2(2) - e^{-2} - 2 \sin(2))}{2 + e^{-2} - 2 \cos(2)} \approx 1.3105$$

Iteration 2:

$$x^{(2)} = x^{(1)} - \frac{f'(x^{(1)})}{f''(x^{(1)})} \approx 1.3105 - \frac{(2(1.3105) - e^{-1.3105} - 2 \sin(1.3105))}{2 + e^{-1.3105} - 2 \cos(1.3105)} \approx 1.0719$$

Iteration 3:

$$x^{(3)} = x^{(2)} - \frac{f'(x^{(2)})}{f''(x^{(2)})} \approx 1.0719 - \frac{(2(1.0719) - e^{-1.0719} - 2 \sin(1.0719))}{2 + e^{-1.0719} - 2 \cos(1.0719)} \approx 1.0393$$

Thus after 3 iterations, we get the approximate minimizer $x^{(3)} \approx 1.0393$